A linear page table with 32-bit address space (2^{32} bytes) & 4kB (2^{12} bytes) pages & 4 byte page table entry takes up 4MB in size.

Further, we have a page table per process.

CRUX: HOW TO MAKE PAGE TABLES SMALLER?

Simple array-based page tables (usually called linear page tables) are too big, taking up far too much memory on typical systems. How can we make page tables smaller? What are the key ideas? What inefficiencies arise as a result of these new data structures?

Simple Solution: Bigger Pages

↳ Take 32 bit address space again but with 16kB pages.

We would thus have:

↳ 18 bit VPN $\frac{2^{32}}{2^{14}} = 2^{18}$

↳ 14 bit offset

Assuming 4 byte 4 byte page table entry, we have

$2^{18} \times 2^2 = 2^{20} = 1MB$

Cons:

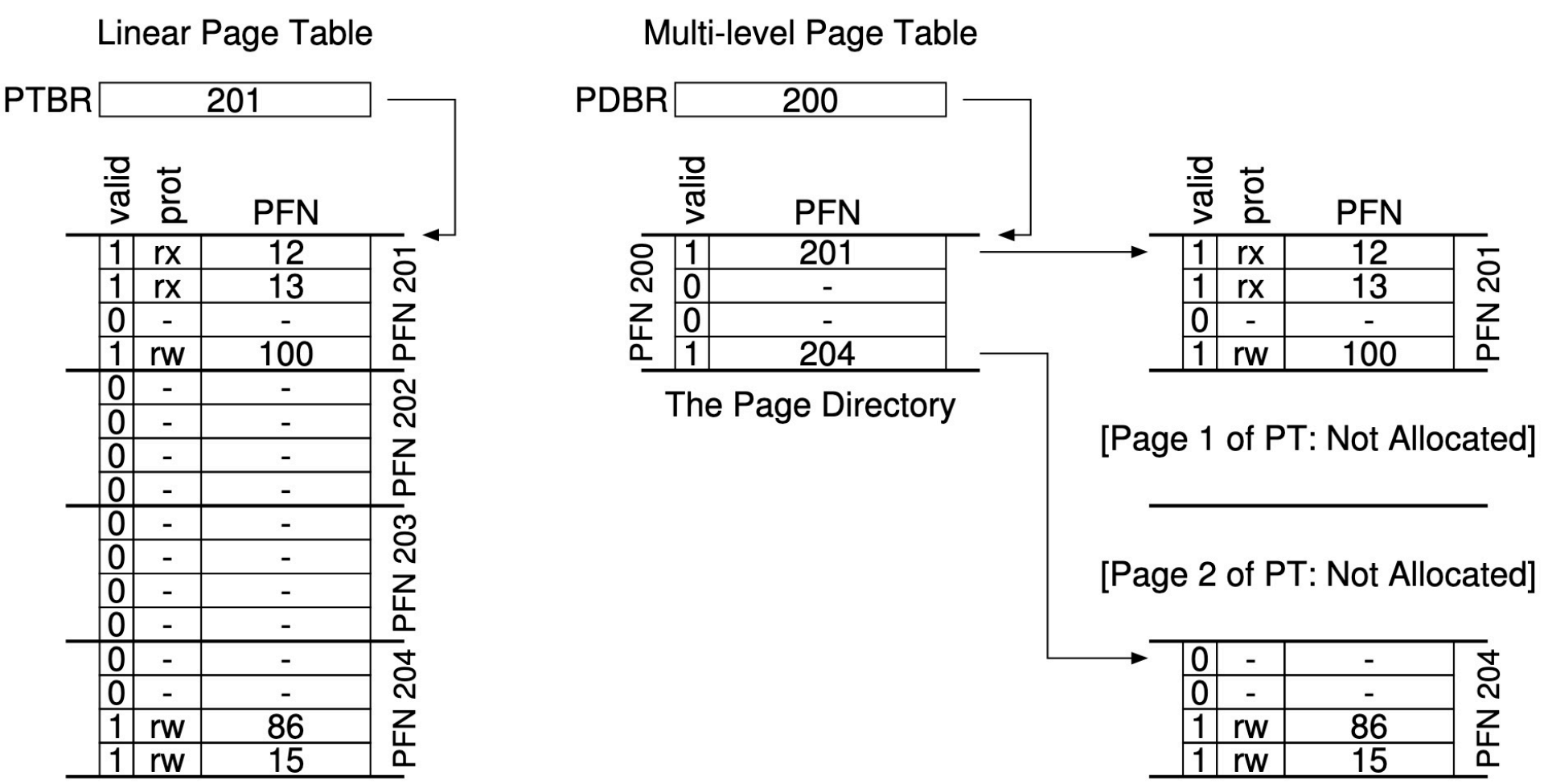
↳ Big pages lead to waste within page i.e internal fragmentation

Multi-level Page table:

↳ First, chop up page table into page-sized units; then,

if an entire page of page-table entries is invalid, don't allocate that page.

To track whether a page of the page table is valid (and if valid, where it is in memory), we use page directory.



- On left, even though most of the middle regions of the address space are not valid, we still allocate the entire page table.
- On the right, page directory marks only two pages of page table as valid, thus only those two reside in memory.
- Page directory is a two-level table, contains one entry per page of page table.

It consists of a no. of Page directory entries (PDE)

A PDE has a valid bit

- If PDE is valid, it means that atleast one of the pages of the page table pointed to by the PDE is valid i.e set to 1
- If PDE is invalid i.e 0 then rest of the PDE is not defined i.e that page in page table isn't allocated.

Advantages:

1. Allocates only page-table space that are being used
2. If constructed carefully, each portion of Page table fits

neatly within a page frame, making it easier to manage free space.

When OS needs to allocate or grow a page table, it can simply grab the next free page frame.

3. Linear page table enforced that entire page table must reside contagiously in memory.

With multi-level structure, page-table pages can be placed anywhere in physical memory.

Disadvantages:

1. On TLB miss, two loads from memory will be required from memory to get the right address.
2. Complexity.

A Detailed Multi-level Example,

Assume address space size 16kB with 64 byte pages.
Thus, we have:

- $\frac{2^{14}}{2^6} = 2^8 \rightarrow$ 8 bit VPN
- 6 bit offset

In this example, VPN 0,1 are for code

VPN 4,5 for heap, 2

VPN 254,255 for stack

0000 0000	code
0000 0001	code
0000 0010	(free)
0000 0011	(free)
0000 0100	heap
0000 0101	heap
0000 0110	(free)
0000 0111	(free)
.....	... all free ...
1111 1100	(free)
1111 1101	(free)
1111 1110	stack
1111 1111	stack

Figure 20.4: **A 16KB Address Space With 64-byte Pages**

Given that our page table has 256 (2^8) entries with each page entry being 4 bytes, our page table size is $2^8 \times 2^2 = 2^{10}$

with 64 byte pages,

$$\frac{2^{10}}{2^6} = 2^4$$

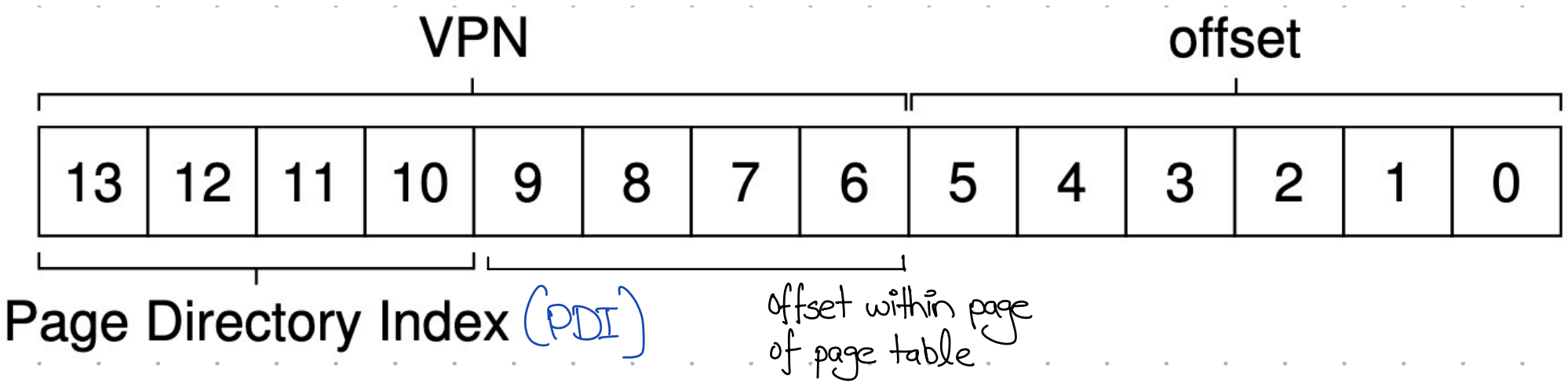
our page table is divided into 16 pages. Thus, page directory will have 2^4 entries i.e we need 4 bits to index into the page directory.

With 64-byte pages & 4 byte per entry:

$$\frac{2^6}{2^2} = 2^4$$

Each page will have 16 (2^4) PTE.

Our virtual address is divided as:



We can use the PDI to find address of page-directory entry. If PDE is marked invalid, it raises an exception.

```
PDIndex = (VPN & PD_MASK) >> PD_SHIFT
PDEAddr = PDBR + (PDIndex * sizeof(PDE))
```

↳ Retrieve the PDE

To find desired PTE, we use the remaining bits of VPN called page-table index (PTI) to index into the page table itself

```
PTIndex = (VPN & PT_MASK) >> PT_SHIFT
PTEAddr = (PDE.PFN<<SHIFT) + (PTIndex*sizeof(PTE))
```

↳ Retrieve the PTE

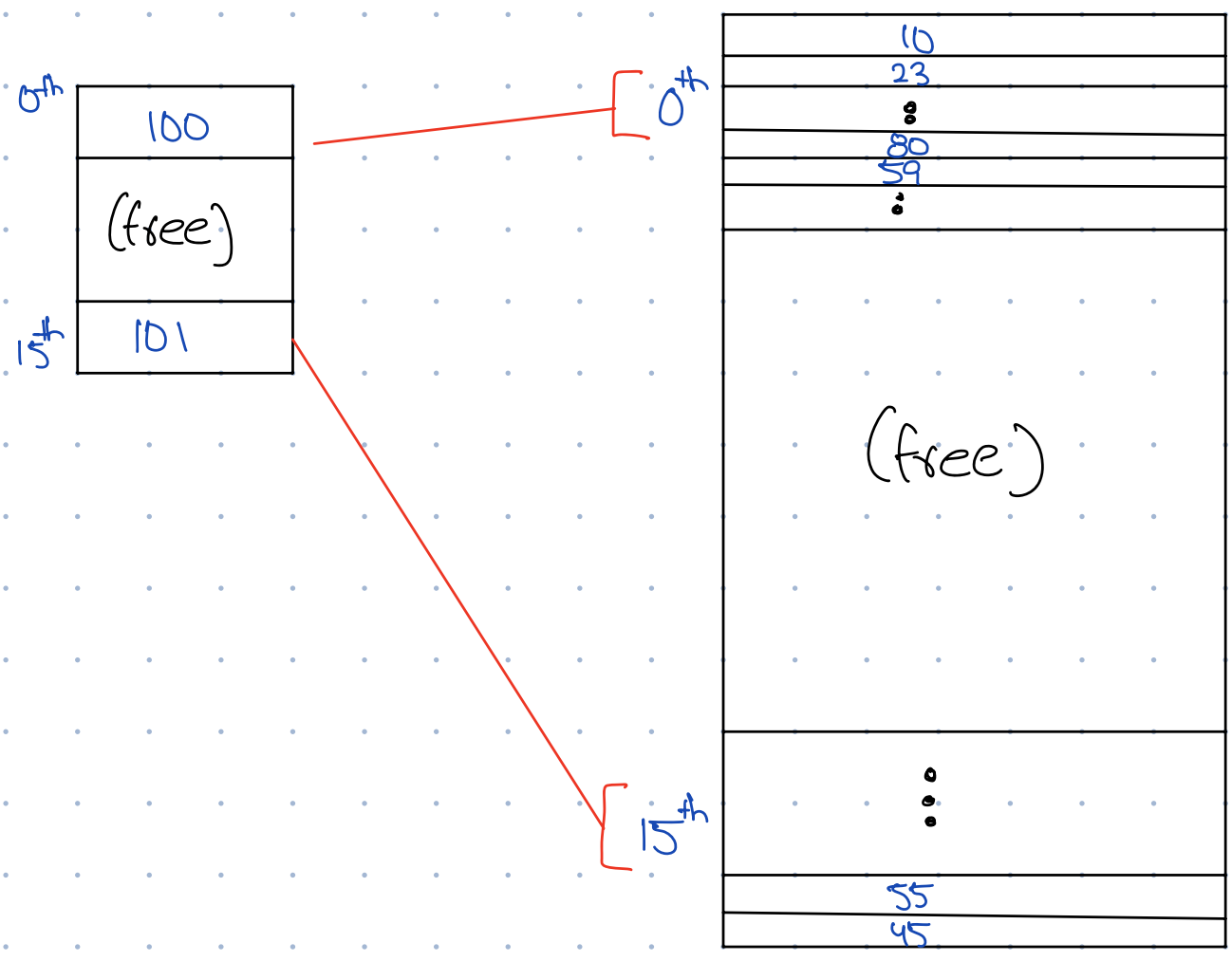
Example with numbers:

Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
PFN	valid?	PFN	valid	prot	PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

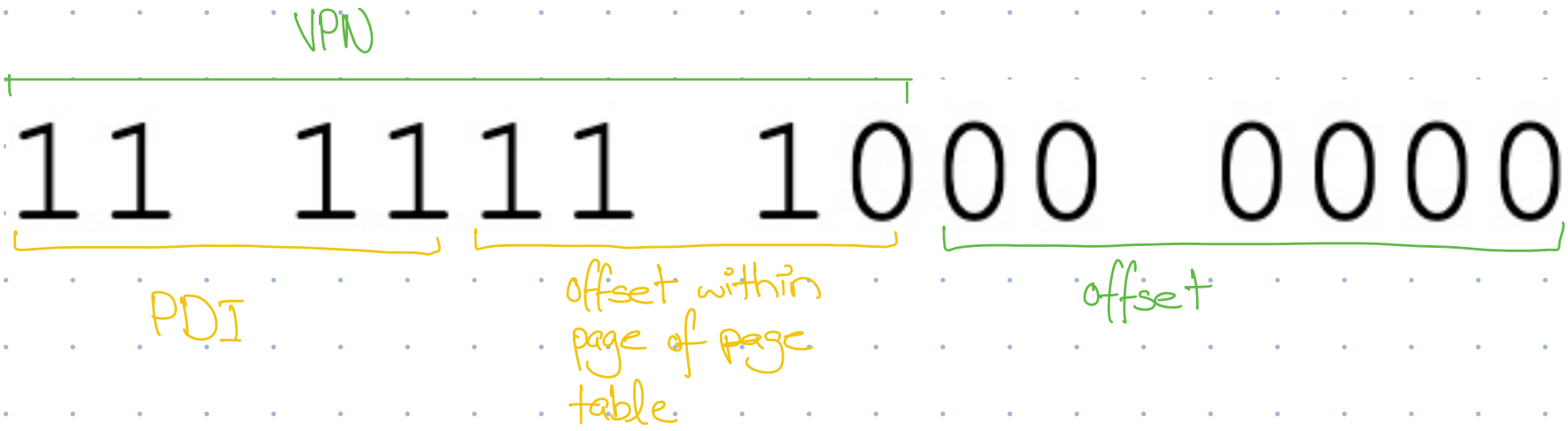
Figure 20.5: **A Page Directory, And Pieces Of Page Table**

0th page of the page table containing 16 PTEs

last page of the page table containing last 16 PTEs



Let our Virtual address be:



Top 4 bits gives us valid page of page table at address 101.

Now,
we use remaining bits i.e 1110 to index into the page of the page table to get our desired PTE i.e 55

We thus form our desired physical address as:

00 1101 1100 0000

Translation process: Remember the TLB

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)   // TLB Hit
4    if (CanAccess(TlbEntry.ProtectBits) == True)
5      Offset   = VirtualAddress & OFFSET_MASK
6      PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7      Register = AccessMemory(PhysAddr)
8    else
9      RaiseException(PROTECTION_FAULT)
10 else                   // TLB Miss
11   // first, get page directory entry
12   PDIndex = (VPN & PD_MASK) >> PD_SHIFT
13   PDEAddr = PDBR + (PDIndex * sizeof(PDE))
14   PDE     = AccessMemory(PDEAddr)
15   if (PDE.Valid == False)
16     RaiseException(SEGMENTATION_FAULT)
17   else
18     // PDE is valid: now fetch PTE from page table
19     PTIndex = (VPN & PT_MASK) >> PT_SHIFT
20     PTEAddr = (PDE.PFN<<SHIFT) + (PTIndex*sizeof(PTE))
21     PTE     = AccessMemory(PTEAddr)
22     if (PTE.Valid == False)
23       RaiseException(SEGMENTATION_FAULT)
24     else if (CanAccess(PTE.ProtectBits) == False)
25       RaiseException(PROTECTION_FAULT)
26     else
27       TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
28       RetryInstruction()
```

Figure 20.6: **Multi-level Page Table Control Flow**